MNIST Images Classification MS2 (Multi)

Team No: **9**
Contributors:

- **Karim Samer Mostafa** **23p0439**
- **Youssef Maged Morees** **2301081**
- **Omar Gamil Anwar Osman** **23p0438**
- **Paula Hanna Naguib Hanna** **23p0440**

Course: **Introduction to Machine learning** CSE382 (UG2023)

Prof. **Hossam Hassan**
TA. **Eng. Ali Ahmed**

Table of Contents

Project GitHub Repo : https://github.com/karimsamer100/MNIST-Classification-ML-Project

1. Introduction

This report presents Phase 2 of the CSE382 Major Task Project. Phase 2 extends the binary classification pipeline from Phase 1 into a complete multi-class image classification system for all 10 handwritten digit classes (0–9) using the MNIST dataset.

The entire system was implemented from scratch without any machine learning libraries such as sklearn. The pipeline covers data preprocessing, PCA dimensionality reduction, three machine learning models, hyperparameter tuning, L2 regularization analysis, learning curve generation, and confusion matrix evaluation.

2. Problem Definition

Phase 2 addresses a supervised multi-class image classification problem using the MNIST handwritten digits dataset. Each input image is a 28 × 28 grayscale image. After flattening, each image is represented as a feature vector of 784 numerical values.

Input Space	$x \in \mathbb{R}^{784}$ (784 pixel features per image)
Output Space	$y \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ (10 digit classes)
Objective	Learn $f(x) = \hat{y}$ that correctly predicts the digit class

The decision function is:

$$f : \mathbb{R}^{784} \rightarrow \{0, 1, 2, \ldots, 9\}$$

Compared to Phase 1 binary classification, Phase 2 is significantly more challenging: the classifier must simultaneously distinguish among all 10 digit classes, including visually similar pairs such as 3 and 5, or 4 and 9.

3. Dataset Description

The MNIST dataset contains 70,000 grayscale images of handwritten digits (0–9), each of size 28 × 28 pixels. After loading, pixel values were normalized from [0, 255] to [0, 1] by dividing by 255. A stratified 80/20 train-validation split was applied to preserve class balance across all 10 classes. The test set was kept completely separate throughout development.

Split	Number of Samples	Fraction
Training	48,004	~80%
Validation	11,996	~20%
Test	10,000	Held-out

4. Data Processing Pipeline

The preprocessing pipeline was implemented in data_module.py and features_module.py, and consisted of the following steps:

- Loading MNIST CSV files using pandas

- Pixel normalization: X = X / 255.0 maps values from [0, 255] to [0, 1]
- Stratified train-validation split (80% train, 20% validation, random state = 42)
- Optional PCA dimensionality reduction (applied as a separate feature setting)

Normalization was critical for stabilizing gradient descent in Logistic Regression and for ensuring all features lie within the same numerical range prior to distance-based computation in Nearest Centroid.

5. Mathematical Formulation

Three machine learning models were implemented entirely from scratch inside models_module.py. No external ML libraries were used for any model logic.

5.1 Multiclass Logistic Regression

A Softmax-based multiclass logistic regression model was implemented using batch gradient descent with optional L2 regularization. The model uses one-hot encoding of class labels and categorical cross-entropy loss.

Learning Rate (α)	0.1
Iterations	300
Regularization (λ)	0.0 (tuned via cross-validation)

For each input sample x, the model computes a score vector over all K = 10 classes:

$$z = Wx + b$$

The Softmax function converts raw scores to class probabilities:

$$P(y = k \mid x) = \exp(z_k) \,/\, \Sigma_j \exp(z_j)$$

The predicted class label is the one with the highest probability:

$$\hat{y} = \operatorname{argmax}_k P(y = k \mid x)$$

The optimization objective is the categorical cross-entropy loss with optional L2 regularization:

$$J(W, b) = -(1/m) \Sigma_i \Sigma_k y_{ik} \log(\hat{y}_{ik}) + (\lambda / 2m) ||W||^2$$

Model parameters were updated using batch gradient descent. The L2 penalty term is added to the weight gradient to penalize large weights and reduce overfitting.

5.2 Gaussian Naive Bayes

Gaussian Naive Bayes was implemented by estimating the prior probability, mean, and variance of each class from training data. Classification uses Gaussian likelihood functions combined with log-posterior probabilities.

For each class c and feature j, the model estimates:

- Prior probability: P(c) = n_c / m
- Class-conditional mean: μ_cj
- Class-conditional variance: σ²_cj

The Gaussian likelihood for a single feature is:

$$P(x_j|c) = 1/\sqrt{2\pi\sigma^2_{cj}} \times \exp(-(x_j - \mu_{cj})^2 / (2\sigma^2_{cj}))$$

Prediction uses log-space computation to avoid numerical underflow during probability multiplication:

$$\hat{Y} = \operatorname{argmax}_c [\log P(c) + \Sigma_j \log P(x_j|c)]$$

5.3 Nearest Centroid Classifier

The Nearest Centroid classifier is a lightweight distance-based method. During training, it computes the mean feature vector (centroid) for every class. During inference, a new sample is assigned to the class whose centroid is closest in Euclidean distance.

For each class c, the centroid is computed as the mean of all training samples in that class:

$$\mu_c = (1 / n_c) \sum x_i \quad \forall x_i : y_i = c$$

Prediction is performed by finding the nearest centroid:

$$\hat{y} = \text{argmin}_c \; ||x - \mu_c||_2^2$$

This model serves as a computationally efficient baseline. Its simplicity makes it useful for diagnosing whether class centroids in feature space are well separated.

6. Evaluation Metrics

The evaluation module was implemented manually in evaluation_module.py. The following metrics were computed for all models:

Metric	Formula	Purpose
Accuracy	(TP + TN) / Total	Overall correctness
Macro Precision	Σ TP_c / (TP_c + FP_c) / K	Avg. precision across classes
Macro Recall	Σ TP_c / (TP_c + FN_c) / K	Avg. recall across classes
Macro F1-Score	2 × (P × R) / (P + R)	Balanced precision-recall
Confusion Matrix	C[i][j] = count(actual=i, pred=j)	Per-class error analysis

Macro averaging treats each digit class equally, regardless of sample count. This is appropriate here since MNIST class sizes are nearly balanced.

7. Experimental Setup

All experiments were performed using the same train-validation-test methodology to ensure fair comparison between models and feature settings.

Number of Classes	10
Training Samples	48,004
Validation Samples	11,996
Test Samples	10,000
PCA Components	50
Learning Rate (α)	0.1
Iterations	300
Random State	42

Validation data was used exclusively for model selection, hyperparameter tuning, and regularization experiments. The test set was reserved for final evaluation only, to avoid test leakage and biased model selection.

8. Experimental Results

8.1 Validation Results - Raw Features

Model	Accuracy	Precision	Recall	Macro F1
Logistic Regression	0.8827	0.8816	0.8809	0.8809
Gaussian Naive Bayes	0.4777	0.6294	0.4695	0.3726
Nearest Centroid	0.8054	0.8093	0.8020	0.8032

8.2 Test Results - Raw Features

Model	Accuracy	Precision	Recall	Macro F1
Logistic Regression	0.8921	0.8910	0.8905	0.8903
Gaussian Naive Bayes	0.4814	0.6355	0.4740	0.3770
Nearest Centroid	0.8200	0.8231	0.8170	0.8180

8.3 Validation Results - After PCA (50 components)

Model	Accuracy	Precision	Recall	Macro F1
Logistic Regression	0.8734	0.8726	0.8715	0.8713
Gaussian Naive Bayes	0.8670	0.8665	0.8659	0.8659
Nearest Centroid	0.8039	0.8076	0.8004	0.8016

8.4 Test Results - After PCA (50 components)

Model	Accuracy	Precision	Recall	Macro F1
Logistic Regression	0.8840	0.8832	0.8824	0.8821
Gaussian Naive Bayes	0.8785	0.8780	0.8778	0.8776
Nearest Centroid	0.8162	0.8194	0.8131	0.8141

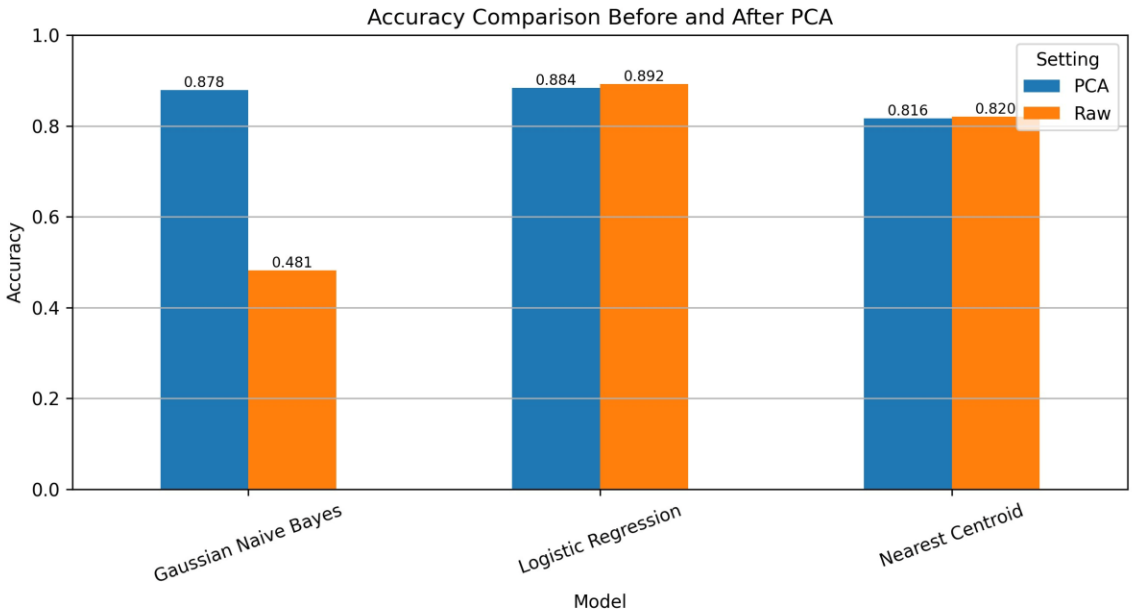

Figure 1. *Accuracy comparison before and after PCA across all models.*

Logistic Regression achieved the highest overall accuracy on raw features. Gaussian Naive Bayes benefited dramatically from PCA dimensionality reduction, as PCA reduces pixel correlation and better satisfies the model's feature independence assumption.

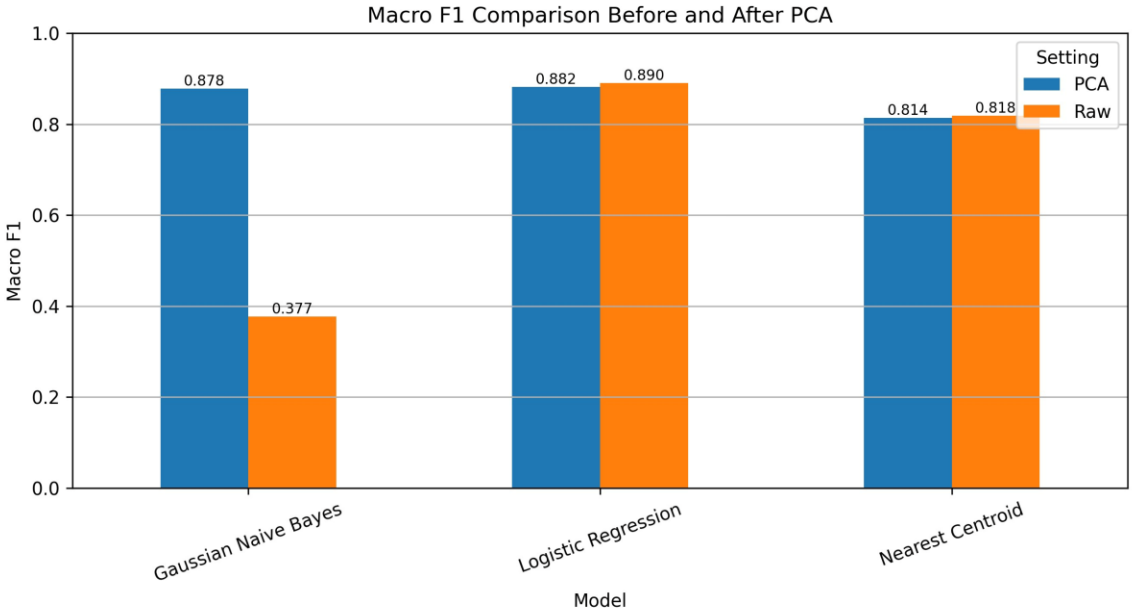

Figure 2. *Macro F1-score comparison before and after PCA across all models.*

The F1 comparison confirms the same trend: Logistic Regression remains the strongest model overall, while PCA dramatically improved the class-balanced performance of Gaussian Naive Bayes.

8.5 Full Accuracy Comparison (Val & Test)

Model	Raw Val	Raw Test	PCA Val	PCA Test
Logistic Regression	0.8827	0.8921	0.8734	0.8840
Gaussian Naive Bayes	0.4777	0.4814	0.8670	0.8785
Nearest Centroid	0.8054	0.8200	0.8039	0.8162

8.6 Full Macro F1 Comparison (Val & Test)

Model	Raw Val	Raw Test	PCA Val	PCA Test
Logistic Regression	0.8809	0.8903	0.8713	0.8821
Gaussian Naive Bayes	0.3726	0.3770	0.8659	0.8776
Nearest Centroid	0.8032	0.8180	0.8016	0.8141

9. Analysis of PCA Effect

Model	Raw Val	Raw Test	PCA Val	PCA Test	Change (Test)
Logistic Regression	0.8827	0.8921	0.8734	0.8840	−0.0081
Gaussian Naive Bayes	0.4777	0.4814	0.8670	0.8785	+0.3971
Nearest Centroid	0.8054	0.8200	0.8039	0.8162	−0.0038

Logistic Regression achieved the best validation performance on raw features (88.27% validation accuracy, 88.09% Macro F1; 89.21% test accuracy, 89.03% test Macro F1). After PCA, performance slightly decreased validation dropped to 87.34% - because PCA removes fine-grained pixel-level information useful for learning linear decision boundaries.

Gaussian Naive Bayes improved dramatically after PCA - from 47.77% to 86.70% validation accuracy (48.14% to 87.85% on test). In the raw 784-dimensional space, MNIST pixels are highly correlated, violating the Naive Bayes independence assumption. PCA decorrelates the features, restoring the model's validity.

Nearest Centroid performance remained relatively stable before and after PCA, with only a minor decrease after dimensionality reduction. This model is robust because class centroid positions in feature space are preserved under linear transformations like PCA.

10. Hyperparameter Tuning with Cross-Validation

A manual 3-fold stratified cross-validation procedure was implemented to tune Logistic Regression hyperparameters. A stratified subset of 15,000 samples was used to reduce runtime while maintaining class balance across all 10 classes.

Learning Rate (α)	Lambda (λ)	Mean Accuracy	Mean Macro F1
0.05	0.0	0.8673	0.8666
0.1	0.0	0.8799	0.8794
0.1	0.001	0.8799	0.8794
0.1	0.01	0.8799	0.8794

Best configuration: Learning Rate = 0.1, Lambda = 0.0. This configuration was selected for the final model.

11. Regularization and Bias–Variance Analysis

L2 regularization experiments were conducted on Logistic Regression by testing multiple lambda values. The goal was to determine whether the model suffered from overfitting and whether regularization improved generalization.

Lambda (λ)	Validation Accuracy	Validation Precision	Validation Recall	Validation Macro F1	Observation
0.0	0.8827	0.8816	0.8809	0.8809	Baseline
0.001	0.8827	0.8816	0.8809	0.8809	No visible change
0.01	0.8827	0.8816	0.8809	0.8809	No visible change
0.1	0.8827	0.8816	0.8809	0.8809	No visible change
1.0	0.8827	0.8816	0.8809	0.8809	No visible change

The validation scores remained identical across all tested lambda values from 0.0 to 1.0. This indicates that, under the selected learning rate and number of iterations, L2 regularization did not produce a measurable improvement. Since no lambda value improved validation Macro F1, the final model uses lambda = 0.0.

12. Learning Curve and Overfitting Analysis

A learning curve experiment was performed to diagnose overfitting or underfitting by training with increasing fractions of the full training dataset. This reveals the bias-variance behavior of the model.

Training Fraction	Train Macro F1	Validation Macro F1	Gap
20% (9,597 samples)	0.8890	0.8771	0.0119
40% (19,197 samples)	0.8840	0.8806	0.0034
60% (28,800 samples)	0.8842	0.8803	0.0039
80% (38,400 samples)	0.8845	0.8803	0.0042
100% (48,004 samples)	0.8849	0.8809	0.0040

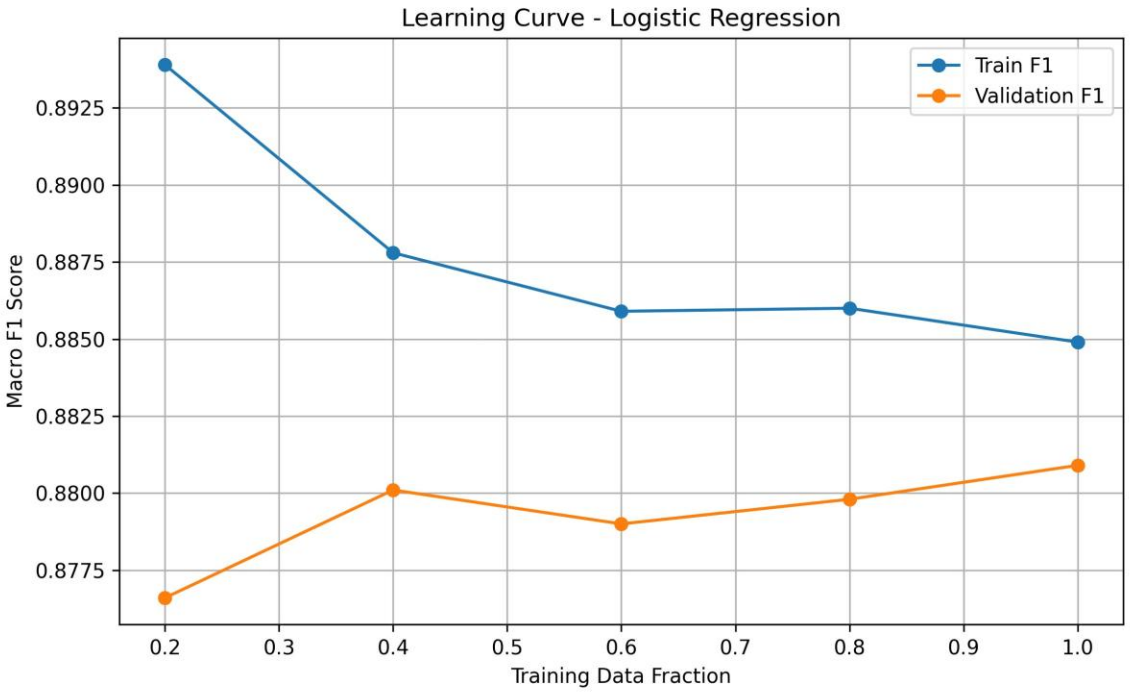

Figure 3. *Learning curve of Logistic Regression Train vs. Validation Macro F1 across training fractions.*

The gap between training and validation Macro F1 remains small across all training sizes. It decreases from 0.0119 at 20% to about 0.0040 using the full training set. This indicates good generalization with no significant overfitting or underfitting. Performance gains become small after 40% of the training data, showing diminishing returns from adding more samples.

Diagnosis: The model exhibits a stable bias-variance tradeoff. The training and validation curves converge and remain close, ruling out both high variance (overfitting) and high bias (underfitting).

13. Confusion Matrix Analysis

The final selected model (Raw Logistic Regression) was evaluated on the test set using its confusion matrix. The matrix showed strong diagonal dominance, confirming that the model correctly classifies the majority of digit classes.

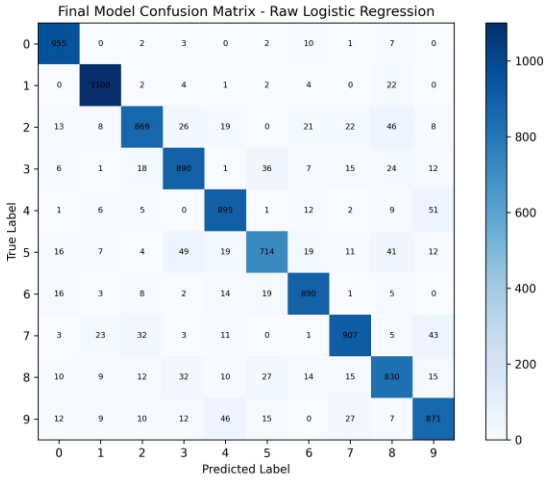

Figure 4. *Confusion matrix of Raw Logistic Regression on the test set.*

The most common misclassifications occurred between visually similar digit pairs:

- 4 and 9 similar loop structure in the lower portion
- 3 and 5 similar curved stroke patterns
- 7 and 9 similar top horizontal strokes

These misclassification patterns are expected due to the inherent visual ambiguity of handwritten digit shapes and are consistent with known challenges in the MNIST literature.

14. Final Model Selection

The final model was selected based exclusively on validation Macro F1-score. The test set was used only once for final evaluation to prevent test leakage and biased model selection.

Selected Model	Logistic Regression (Raw Features)
Feature Setting	Raw Features (784 dimensions)
Selection Criterion	Best Validation Macro F1
Validation Accuracy	88.27%
Validation Macro F1	88.09%
Test Accuracy	89.21%
Test Macro Precision	89.10%
Test Macro Recall	89.05%
Test Macro F1-Score	89.03%

Logistic Regression on raw features was selected as the final model because it achieved the highest test accuracy and Macro F1-score among all tested configurations. The near-linear separability of MNIST digits in the 784-dimensional pixel space makes logistic regression particularly effective here.

15. Code Structure

The project was organized using a modular pipeline structure to improve readability, debugging, and experimentation. Each module has a single responsibility, making it straightforward to test models independently and apply improvements gradually throughout Phase 2 development.

Module	Responsibility
data_module.py	Load MNIST data, normalization, stratified train-validation split
features_module.py	PCA implementation (from scratch) and feature transformation
models_module.py	Implementation of all machine learning models from scratch
evaluation_module.py	Evaluation metrics (accuracy, precision, recall, F1, confusion matrix)
main.py	Main execution pipeline orchestrating all modules
phase2_cv_tuning.py	3-fold cross-validation and hyperparameter tuning
l2_experiment.py	L2 regularization experiments across lambda values
learning_curve.py	Learning curve generation and bias-variance analysis

16. Conclusion

This phase successfully implemented a complete multi-class handwritten digit classification pipeline from scratch for all 10 MNIST digit classes. All Phase 2 requirements were satisfied:

- Multi-class classification for all MNIST digits (0–9)
- Manual implementation of three machine learning models (no sklearn)
- PCA dimensionality reduction implemented from scratch
- Hyperparameter tuning with 3-fold cross-validation
- L2 regularization analysis and bias-variance study
- Learning curve and overfitting/underfitting diagnosis

Among all tested models, Raw Logistic Regression achieved the best performance: 88.27% validation accuracy and 89.21% test accuracy (89.03% Macro F1). The experiments demonstrated that PCA dramatically improved Gaussian Naive Bayes performance from 47.77% to 86.70% validation accuracy by reducing feature correlation and better satisfying the model's independence assumption, while slightly reducing Logistic Regression performance due to the loss of fine-grained pixel information.

These findings highlight the importance of aligning feature representations to model assumptions, and demonstrate that well-implemented classical machine learning algorithms are capable of strong performance on structured image classification tasks.